TELCO CUSTOMER : CLASSIFICATION ANALYSIS

This project focuses on building a customer churn classification model for a telecommunications company using a dataset containing 7,043 customer records and 21 attributes. The objective is to predict whether a customer is likely to churn (leave the service) based on demographic information, account details, and service usage patterns. The dataset includes features such as customer tenure, contract type, payment method, monthly charges, and subscribed services. Through data preprocessing, exploratory data analysis, and the application of machine learning classification algorithms, the project aims to identify key factors influencing churn and improve customer retention strategies. The final model helps the business make data-driven decisions by proactively targeting high-risk customers with personalized offers and retention plans.

IMPORTING SUFFICIENT LIBARIES

In [49]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
```

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier
from xgboost import XGBClassifier
```

IMPORTING DATASET

In [50]:
```
import pandas as pd
df=pd.read_csv('/content/drive/MyDrive/project/WA_Fn-UseC_-Telco-Customer-Chur
df
```

Out[50]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneS
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	
7039	2234-XADUH	Female	0	Yes	Yes	72	
7040	4801-JZAZL	Female	0	Yes	Yes	11	
7041	8361-LTMKD	Male	1	Yes	No	4	
7042	3186-AJIEK	Male	0	No	No	66	

7043 rows × 21 columns

CHECKING DUPLICATES

In [51]:
```
df.drop_duplicates()
```

Out[51]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneS
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	
7039	2234-XADUH	Female	0	Yes	Yes	72	
7040	4801-JZAZL	Female	0	Yes	Yes	11	
7041	8361-LTMKD	Male	1	Yes	No	4	
7042	3186-AJIEK	Male	0	No	No	66	

7043 rows × 21 columns

CHECKING MISSING VALUES

In [52]:
```
df.isna().sum()
```

Out[52]:

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

CHECKING DATA TYPES

In [53]:
```
df.dtypes
```

Out[53]:

	0
customerID	object
gender	object
SeniorCitizen	int64
Partner	object
Dependents	object
tenure	int64
PhoneService	object
MultipleLines	object
InternetService	object
OnlineSecurity	object
OnlineBackup	object
DeviceProtection	object
TechSupport	object
StreamingTV	object
StreamingMovies	object
Contract	object
PaperlessBilling	object
PaymentMethod	object
MonthlyCharges	float64
TotalCharges	object
Churn	object

dtype: object

ENCODING OBJECTS

In [54]:

```
from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()
df["customerID"]=encoder.fit_transform(df["customerID"])
df["gender"]=encoder.fit_transform(df["gender"])
df["Partner"]=encoder.fit_transform(df["Partner"])
df["Dependents"]=encoder.fit_transform(df["Dependents"])
df["PhoneService"]=encoder.fit_transform(df["PhoneService"])
df["MultipleLines"]=encoder.fit_transform(df["MultipleLines"])
df["InternetService"]=encoder.fit_transform(df["InternetService"])
df["OnlineSecurity"]=encoder.fit_transform(df["OnlineSecurity"])
```

```
df["OnlineBackup"]=encoder.fit_transform(df["OnlineBackup"])
df["DeviceProtection"]=encoder.fit_transform(df["DeviceProtection"])
df["TechSupport"]=encoder.fit_transform(df["TechSupport"])
df["StreamingTV"]=encoder.fit_transform(df["StreamingTV"])
df["StreamingMovies"]=encoder.fit_transform(df["StreamingMovies"])
df["Contract"]=encoder.fit_transform(df["Contract"])
df["PaperlessBilling"]=encoder.fit_transform(df["PaperlessBilling"])
df["PaymentMethod"]=encoder.fit_transform(df["PaymentMethod"])
df["TotalCharges"]=encoder.fit_transform(df["TotalCharges"])
df["Churn"]=encoder.fit_transform(df["Churn"])
df
```

Out[54]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneS
0	5375	0	0	1	0	1	
1	3962	1	0	0	0	34	
2	2564	1	0	0	0	2	
3	5535	1	0	0	0	45	
4	6511	0	0	0	0	2	
...	...	...	...	...	...	...	
7038	4853	1	0	1	1	24	
7039	1525	0	0	1	1	72	
7040	3367	0	0	1	1	11	
7041	5934	1	1	1	0	4	
7042	2226	1	0	0	0	66	

7043 rows × 21 columns

In [55]:
```
x=df.iloc[:,:-1]
x
```

Out[55]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneS
0	5375	0	0	1	0	1	
1	3962	1	0	0	0	34	
2	2564	1	0	0	0	2	
3	5535	1	0	0	0	45	
4	6511	0	0	0	0	2	
...	...	...	...	...	...	...	
7038	4853	1	0	1	1	24	
7039	1525	0	0	1	1	72	
7040	3367	0	0	1	1	11	
7041	5934	1	1	1	0	4	
7042	2226	1	0	0	0	66	

7043 rows × 20 columns

SEPARATE FEATURE AND TARGETS

In [56]:

```
y=df.iloc[:,-1]
y
```

Out[56]:

	Churn
0	0
1	0
2	1
3	0
4	1
...	...
7038	0
7039	0
7040	0
7041	1
7042	0

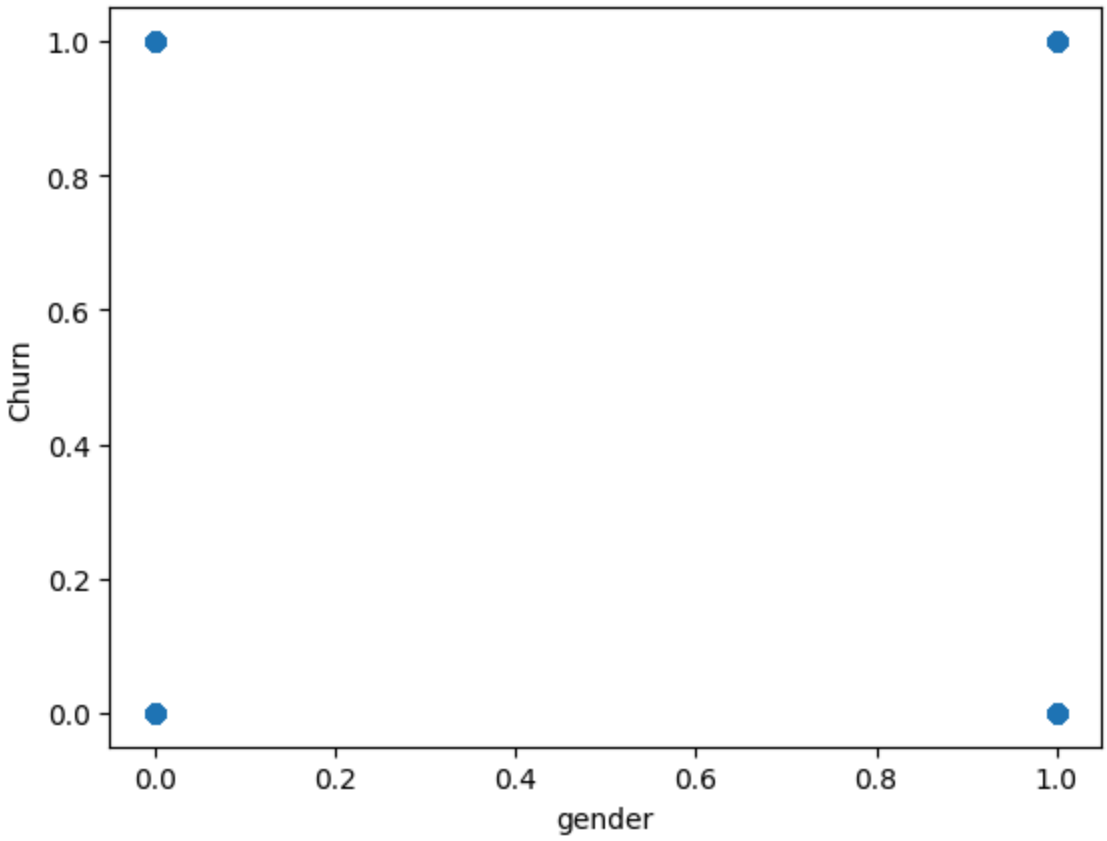
7043 rows × 1 columns

dtype: int64

PLOTTING

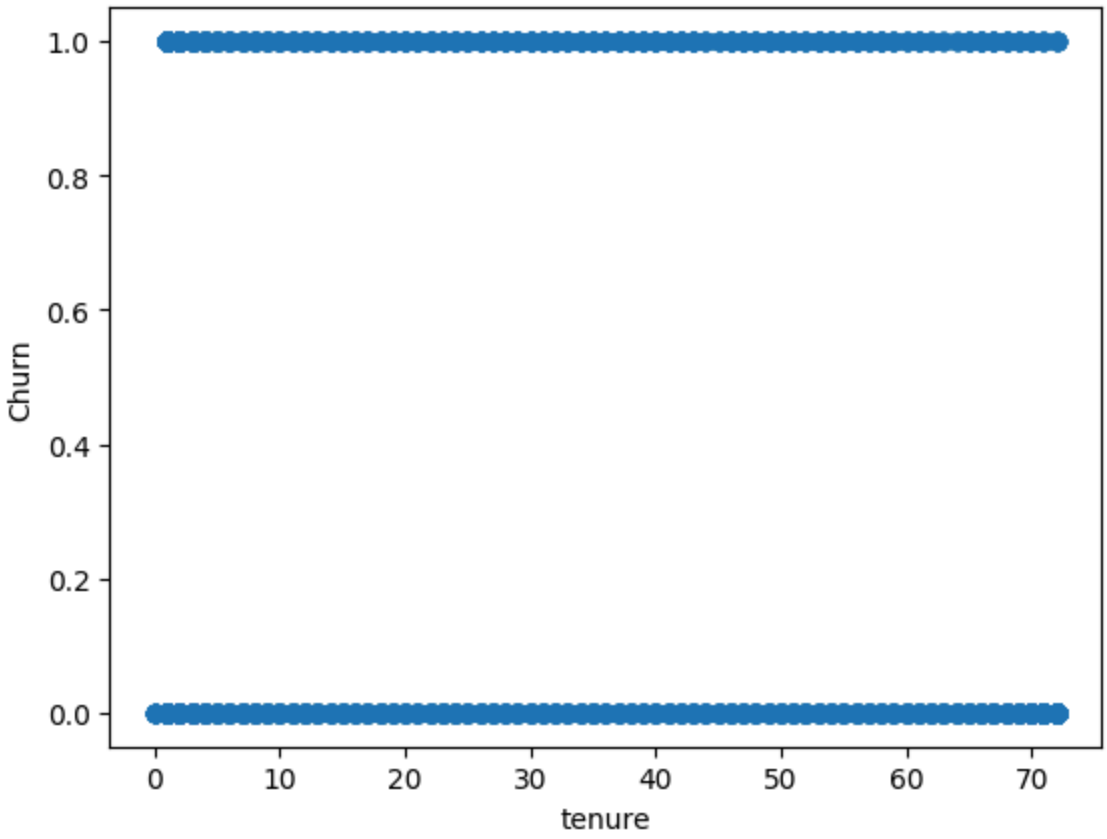
In [57]:
```
import matplotlib.pyplot as plt
plt.scatter(x['gender'],y)
plt.xlabel("gender")
plt.ylabel("Churn")
```

Out[57]:
```
Text(0, 0.5, 'Churn')
```



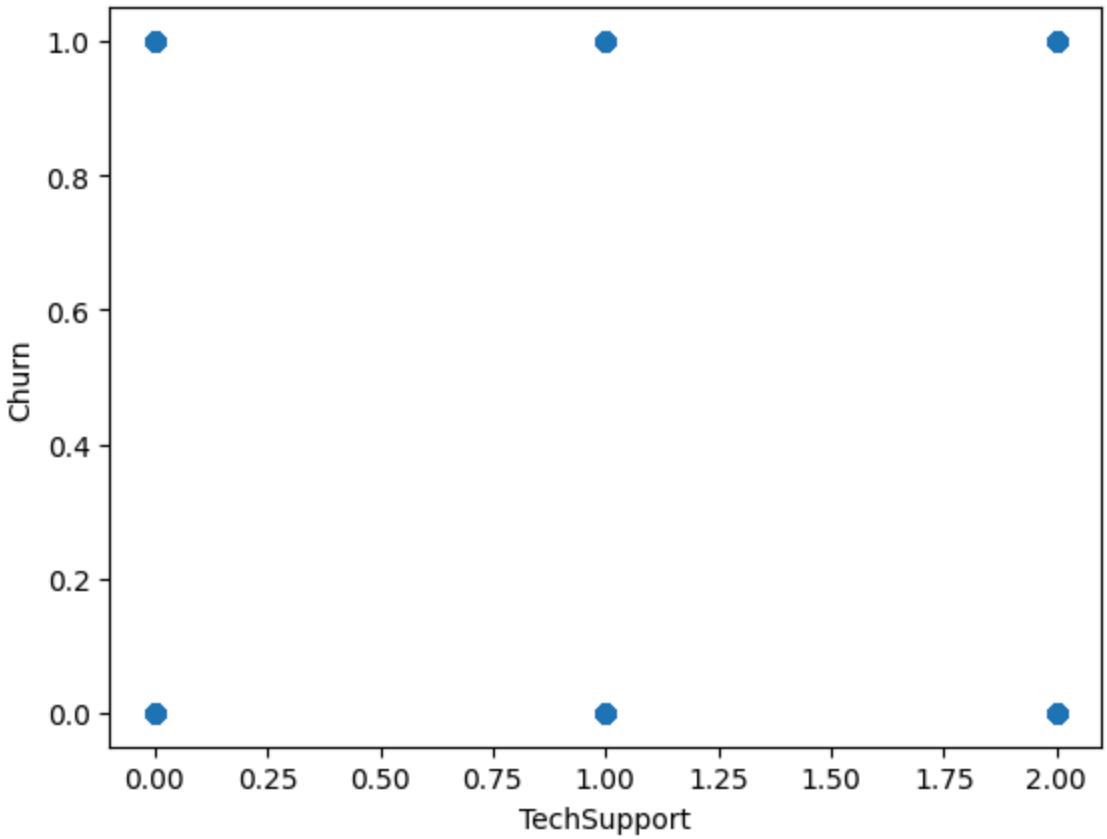
In [58]:
```
import matplotlib.pyplot as plt
plt.scatter(x['tenure'],y)
plt.xlabel("tenure")
plt.ylabel("Churn")
```

Out[58]:
```
Text(0, 0.5, 'Churn')
```



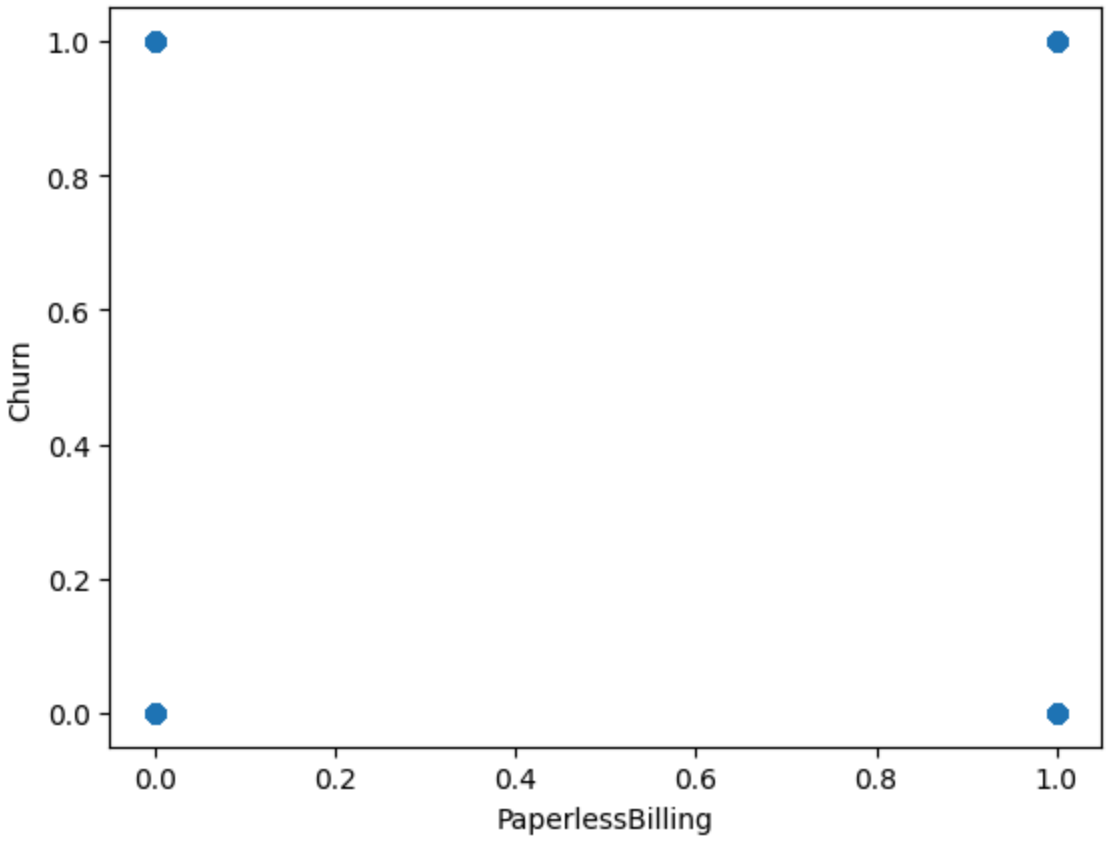
In [59]:
```
import matplotlib.pyplot as plt
plt.scatter(x['TechSupport'],y)
plt.xlabel("TechSupport")
plt.ylabel("Churn")
```

Out[59]:
```
Text(0, 0.5, 'Churn')
```



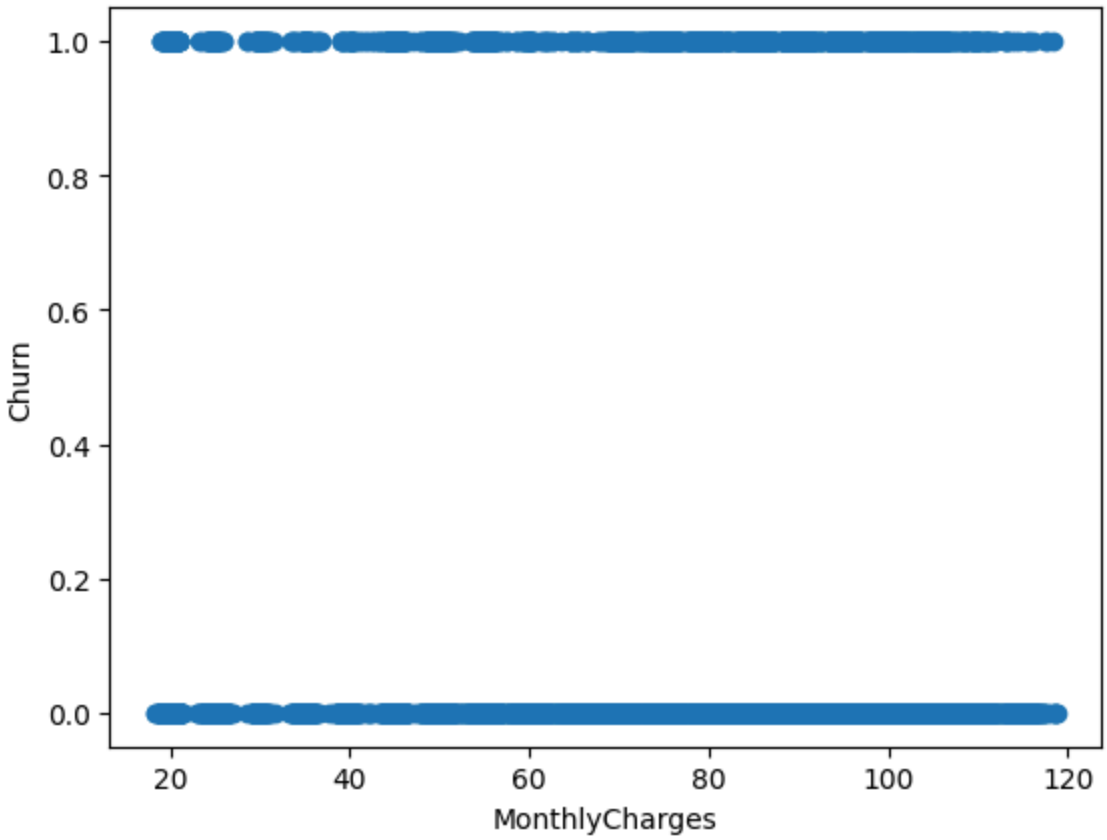
In [60]:
```
import matplotlib.pyplot as plt
plt.scatter(x['PaperlessBilling'],y)
plt.xlabel("PaperlessBilling")
plt.ylabel("Churn")
```

Out[60]:
```
Text(0, 0.5, 'Churn')
```



In [61]:
```
import matplotlib.pyplot as plt
plt.scatter(x['MonthlyCharges'],y)
plt.xlabel("MonthlyCharges")
plt.ylabel("Churn")
```

Out[61]:
```
Text(0, 0.5, 'Churn')
```



SCALING

In [62]:
```
from sklearn.preprocessing import MinMaxScaler
scaler=MinMaxScaler()
x_scaled = scaler.fit_transform(x)
x_scaled
```

Out[62]:
```
array([[0.76327748, 0.        , 0.        , ..., 0.66666667, 0.11542289,
        0.38361409],
       [0.56262425, 1.        , 0.        , ..., 1.        , 0.38507463,
        0.2245023 ],
       [0.36410111, 1.        , 0.        , ..., 1.        , 0.35422886,
        0.02404288],
       ...,
       [0.47813121, 0.        , 0.        , ..., 0.66666667, 0.11293532,
        0.45849923],
       [0.84265834, 1.        , 1.        , ..., 1.        , 0.55870647,
        0.40735069],
       [0.31610338, 1.        , 0.        , ..., 0.        , 0.86965174,
        0.8280245 ]])
```

TRAIN TEST SPLIT

In [63]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x_scaled,y,test_size=0.3,rand
```

In [64]:
```
x_train.shape,y_train.shape
```

Out[64]:
```
((4930, 20), (4930,))
```

In [65]:
```
x_test.shape,y_test.shape
```

Out[65]:
```
((2113, 20), (2113,))
```

MODEL CREATION

In [66]:
```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
knn=KNeighborsClassifier(n_neighbors=5)
sv=SVC()
nb=GaussianNB()
dt=DecisionTreeClassifier(criterion='entropy')
```

In [67]:
```
models=[knn,sv,nb,dt]
for model in models:
  print(model)
  model.fit(x_train,y_train)
  y_pred=model.predict(x_test)
  print(classification_report(y_test,y_pred))
```

```
KNeighborsClassifier()
              precision    recall  f1-score   support

           0       0.84      0.84      0.84      1585
           1       0.52      0.51      0.52       528

    accuracy                           0.76      2113
   macro avg       0.68      0.68      0.68      2113
weighted avg       0.76      0.76      0.76      2113

SVC()
              precision    recall  f1-score   support

           0       0.84      0.91      0.88      1585
           1       0.65      0.50      0.56       528

    accuracy                           0.81      2113
   macro avg       0.75      0.70      0.72      2113
weighted avg       0.80      0.81      0.80      2113

GaussianNB()
              precision    recall  f1-score   support

           0       0.91      0.76      0.83      1585
           1       0.52      0.76      0.62       528

    accuracy                           0.76      2113
   macro avg       0.71      0.76      0.72      2113
weighted avg       0.81      0.76      0.78      2113

DecisionTreeClassifier(criterion='entropy')
              precision    recall  f1-score   support

           0       0.85      0.81      0.83      1585
           1       0.50      0.56      0.53       528

    accuracy                           0.75      2113
   macro avg       0.67      0.68      0.68      2113
weighted avg       0.76      0.75      0.75      2113
```

In [68]:
```
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier,Gradier
from xgboost import XGBClassifier
rf=RandomForestClassifier(random_state=42)
ad=AdaBoostClassifier()
gd=GradientBoostingClassifier()
xg=XGBClassifier()
```

In [69]:
```
from sklearn.metrics import classification_report
models=[rf,ad,gd,xg]
for model in models:
  print('******',model,'******')
  model.fit(x_train,y_train)
```

```
    y_pred = model.predict(x_test)
    print(classification_report(y_test,y_pred))
```

```
****** RandomForestClassifier(random_state=42) ******
              precision    recall  f1-score   support

           0       0.85      0.89      0.87      1585
           1       0.63      0.53      0.58       528

    accuracy                           0.80      2113
   macro avg       0.74      0.71      0.72      2113
weighted avg       0.80      0.80      0.80      2113

****** AdaBoostClassifier() ******
              precision    recall  f1-score   support

           0       0.87      0.88      0.87      1585
           1       0.63      0.60      0.61       528

    accuracy                           0.81      2113
   macro avg       0.75      0.74      0.74      2113
weighted avg       0.81      0.81      0.81      2113

****** GradientBoostingClassifier() ******
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1585
           1       0.65      0.56      0.60       528

    accuracy                           0.81      2113
   macro avg       0.75      0.73      0.74      2113
weighted avg       0.81      0.81      0.81      2113

****** XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=Non
e,
              n_jobs=None, num_parallel_tree=None, ...) ******
              precision    recall  f1-score   support

           0       0.86      0.88      0.87      1585
           1       0.61      0.56      0.58       528

    accuracy                           0.80      2113
   macro avg       0.73      0.72      0.73      2113
weighted avg       0.80      0.80      0.80      2113
```

SAMPLING - OVER SAMPLING

In [70]:
```
y.value_counts()
```

Out[70]:

Churn	count
0	5174
1	1869

dtype: int64

In [71]:
```
from imblearn.over_sampling import SMOTE
os=SMOTE(random_state=1)
```

In [72]:
```
x_os,y_os=os.fit_resample(x,y)
```

In [73]:
```
x_os.shape
```

Out[73]:
```
(10348, 20)
```

In [74]:
```
x_os_scaled=scaler.fit_transform(x_os)
x_os_scaled
```

Out[74]:
```
array([[0.76327748, 0.        , 0.        , ..., 0.66666667, 0.11542289,
        0.38361409],
       [0.56262425, 1.        , 0.        , ..., 1.        , 0.38507463,
        0.2245023 ],
       [0.36410111, 1.        , 0.        , ..., 1.        , 0.35422886,
        0.02404288],
       ...,
       [0.27179778, 0.        , 0.        , ..., 0.66666667, 0.64119147,
        0.90168453],
       [0.9084067 , 0.        , 0.        , ..., 0.66666667, 0.65317791,
        0.1323124 ],
       [0.50298211, 0.        , 0.        , ..., 0.66666667, 0.50001498,
        0.58683002]])
```

In [75]:
```
x_train_os,x_test_os,y_train_os,y_test_os=train_test_split(x_os_scaled,y_os,ra
```

In [76]:
```
models=[knn,sv,nb]
for model in models:
  print("******",model,"******")
  model.fit(x_train_os,y_train_os)
  y_pred_us=model.predict(x_test_os)
  print(classification_report(y_test_os,y_pred_us))
```

```
****** KNeighborsClassifier() ******
              precision    recall  f1-score   support

           0       0.86      0.72      0.78      1330
           1       0.75      0.88      0.81      1257

    accuracy                           0.80      2587
   macro avg       0.80      0.80      0.80      2587
weighted avg       0.81      0.80      0.80      2587

****** SVC() ******
              precision    recall  f1-score   support

           0       0.84      0.82      0.83      1330
           1       0.82      0.84      0.83      1257

    accuracy                           0.83      2587
   macro avg       0.83      0.83      0.83      2587
weighted avg       0.83      0.83      0.83      2587

****** GaussianNB() ******
              precision    recall  f1-score   support

           0       0.82      0.77      0.80      1330
           1       0.77      0.83      0.80      1257

    accuracy                           0.80      2587
   macro avg       0.80      0.80      0.80      2587
weighted avg       0.80      0.80      0.80      2587
```

In [77]:
```
y_os.value_counts()
```

Out[77]:

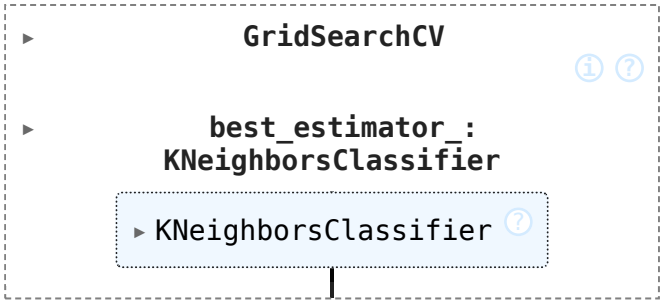
Churn	count
0	5174
1	5174

dtype: int64

HYPER PARAMETER TUNING

In [78]:
```
from sklearn.model_selection import GridSearchCV
params={'n_neighbors':[3,5,7,9],
        'weights':['uniform','distance'],
        'algorithm':['auto','ball_tree','kd_tree','brute']}
clf=GridSearchCV(knn,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[78]:

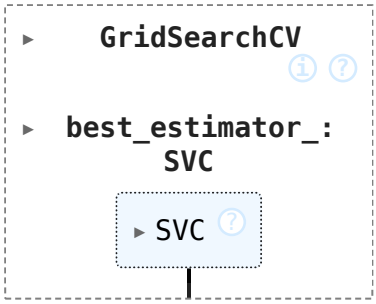


In [79]:
```
print(clf.best_params_)
```

```
{'algorithm': 'auto', 'n_neighbors': 9, 'weights': 'uniform'}
```

In [81]:
```
from sklearn.model_selection import GridSearchCV
params={'C':[1,1.0,10],
        'kernel':['linear','poly','rbf','sigmoid'],
        'gamma':['scale','auto']}
clf=GridSearchCV(sv,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[81]:

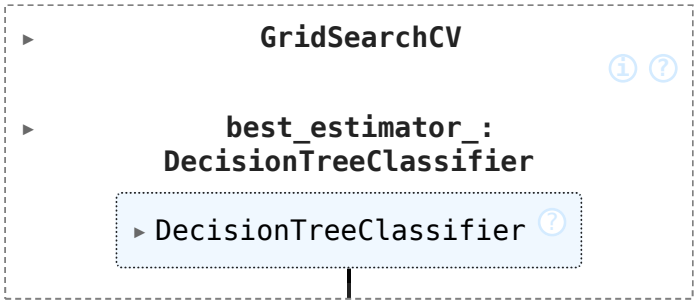


In [82]:
```
print(clf.best_params_)
```

```
{'C': 1, 'gamma': 'scale', 'kernel': 'linear'}
```

In [83]:
```
from sklearn.model_selection import GridSearchCV
params={'criterion':['gini','entropy','log_loss'],
        'splitter':['best','random'],
         'max_depth':[1,2,3,4,5]}
clf=GridSearchCV(dt,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[83]:

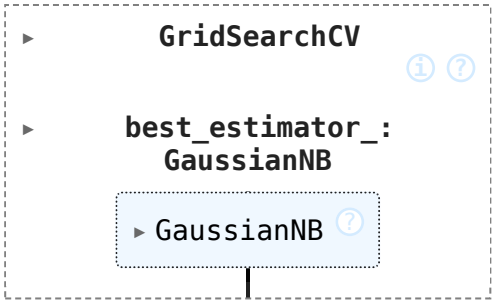


In [84]:
```
print(clf.best_params_)
```

```
{'criterion': 'gini', 'max_depth': 5, 'splitter': 'random'}
```

In [85]:
```
from sklearn.model_selection import GridSearchCV
params={'var_smoothing':[1,1,5,2,2,5,3]}
clf=GridSearchCV(nb,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[85]:

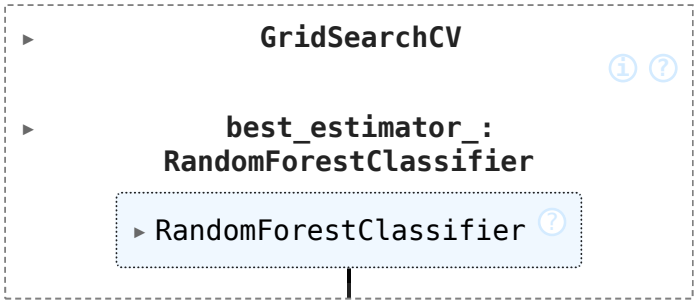


In [86]:
```
print(clf.best_params_)
```

```
{'var_smoothing': 2}
```

In [90]:
```
from sklearn.model_selection import GridSearchCV
params={'n_estimators':[100,101,102,103],
        'criterion':['gini','entropy','log_loss'],
        'max_depth':[1,2,3,4,5]}
clf=GridSearchCV(rf,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[90]:

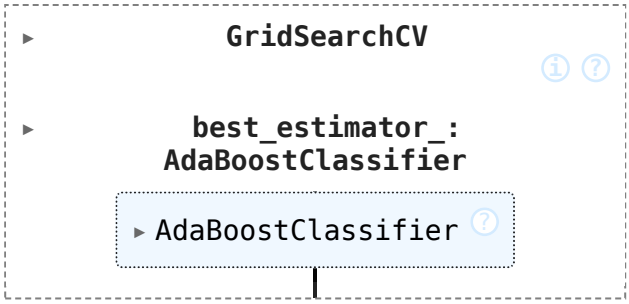


In [91]:
```
print(clf.best_params_)
```

```
{'criterion': 'gini', 'max_depth': 5, 'n_estimators': 103}
```

In [92]:
```
from sklearn.model_selection import GridSearchCV
params={'n_estimators':[100,101,102,103],
        'learning_rate':[1,1,3,4,5,5]}
clf=GridSearchCV(ad,params,cv=10,scoring='accuracy')
clf.fit(x_train,y_train)
```

Out[92]:



In [93]:
```
print(clf.best_params_)
```

```
{'learning_rate': 1, 'n_estimators': 100}
```

PERFOMANCE MEASURE

In [94]:
```
knn1=KNeighborsClassifier(n_neighbors=3,weights='distance',algorithm='auto')
sv1=SVC(C=3,kernel='poly',gamma='scale',degree=5)
nb1=GaussianNB(var_smoothing=1)
dt1=DecisionTreeClassifier(criterion='entropy',splitter='best',max_depth=5)
rf1=RandomForestClassifier(n_estimators=102,criterion='entropy',max_depth=5)
ad1=AdaBoostClassifier(n_estimators=102,learning_rate=1)
```

In [95]:
```
models1=[knn1,sv1,nb1,dt1,rf1,ad1]
for model in models1:
  print(model)
  model.fit(x_train,y_train)
  y_pred=model.predict(x_test)
  print(classification_report(y_test,y_pred))
```

```
KNeighborsClassifier(n_neighbors=3, weights='distance')
              precision    recall  f1-score   support

           0       0.83      0.82      0.83      1585
           1       0.48      0.50      0.49       528

    accuracy                           0.74      2113
   macro avg       0.66      0.66      0.66      2113
weighted avg       0.74      0.74      0.74      2113

SVC(C=3, degree=5, kernel='poly')
              precision    recall  f1-score   support

           0       0.84      0.81      0.82      1585
           1       0.49      0.55      0.52       528

    accuracy                           0.74      2113
   macro avg       0.66      0.68      0.67      2113
weighted avg       0.75      0.74      0.75      2113

GaussianNB(var_smoothing=1)
              precision    recall  f1-score   support

           0       0.88      0.82      0.85      1585
           1       0.55      0.65      0.60       528

    accuracy                           0.78      2113
   macro avg       0.71      0.74      0.72      2113
weighted avg       0.80      0.78      0.79      2113

DecisionTreeClassifier(criterion='entropy', max_depth=5)
              precision    recall  f1-score   support

           0       0.85      0.90      0.87      1585
           1       0.63      0.51      0.56       528

    accuracy                           0.80      2113
   macro avg       0.74      0.70      0.72      2113
weighted avg       0.79      0.80      0.79      2113

RandomForestClassifier(criterion='entropy', max_depth=5, n_estimators=102)
              precision    recall  f1-score   support

           0       0.83      0.92      0.87      1585
           1       0.65      0.45      0.54       528

    accuracy                           0.80      2113
   macro avg       0.74      0.69      0.70      2113
weighted avg       0.79      0.80      0.79      2113

AdaBoostClassifier(learning_rate=1, n_estimators=102)
              precision    recall  f1-score   support

           0       0.86      0.89      0.88      1585
```

```
           1       0.63      0.57      0.60       528

    accuracy                           0.81      2113
   macro avg       0.75      0.73      0.74      2113
weighted avg       0.80      0.81      0.81      2113
```

FEATURE SELECTION

In [96]:
```
df.corr()
```

Out[96]:

	customerID	gender	SeniorCitizen	Partner	Dependents
customerID	1.000000	0.006288	-0.002074	-0.026729	-0.012823
gender	0.006288	1.000000	-0.001874	-0.001808	0.010517
SeniorCitizen	-0.002074	-0.001874	1.000000	0.016479	-0.211185
Partner	-0.026729	-0.001808	0.016479	1.000000	0.452676
Dependents	-0.012823	0.010517	-0.211185	0.452676	1.000000
tenure	0.008035	0.005106	0.016567	0.379697	0.159712
PhoneService	-0.006483	-0.006488	0.008576	0.017706	-0.001762
MultipleLines	0.004316	-0.006739	0.146185	0.142410	-0.024991
InternetService	-0.012407	-0.000863	-0.032310	0.000891	0.044590
OnlineSecurity	0.013292	-0.015017	-0.128221	0.150828	0.152166
OnlineBackup	-0.003334	-0.012057	-0.013632	0.153130	0.091015
DeviceProtection	-0.006918	0.000549	-0.021398	0.166330	0.080537
TechSupport	0.001140	-0.006825	-0.151268	0.126733	0.133524
StreamingTV	-0.007777	-0.006421	0.030776	0.137341	0.046885
StreamingMovies	-0.016746	-0.008743	0.047266	0.129574	0.021321
Contract	0.015028	0.000126	-0.142554	0.294806	0.243187
PaperlessBilling	-0.001945	-0.011754	0.156530	-0.014877	-0.111377
PaymentMethod	0.011604	0.017352	-0.038551	-0.154798	-0.040292
MonthlyCharges	-0.003916	-0.014569	0.220173	0.096848	-0.113890
TotalCharges	0.003027	-0.005291	0.037653	0.059568	-0.009572
Churn	-0.017447	-0.008612	0.150889	-0.150448	-0.164221

21 rows × 21 columns

In [97]:
```
x1=x.drop(['customerID','gender','PhoneService','MultipleLines','InternetServi
x1
```

Out[97]:

	SeniorCitizen	Partner	Dependents	tenure	OnlineSecurity	OnlineBackup
0	0	1	0	1	0	2
1	0	0	0	34	2	0
2	0	0	0	2	2	2
3	0	0	0	45	2	0
4	0	0	0	2	0	0
...	...	...	...	...	...	...
7038	0	1	1	24	2	0
7039	0	1	1	72	0	2
7040	0	1	1	11	2	0
7041	1	1	0	4	0	0
7042	0	0	0	66	2	0

7043 rows × 12 columns

In [98]:
```
x1_scaled=scaler.fit_transform(x1)
x1_scaled
```

Out[98]:
```
array([[0.        , 1.        , 0.        , ..., 1.        , 0.66666667,
        0.11542289],
       [0.        , 0.        , 0.        , ..., 0.        , 1.        ,
        0.38507463],
       [0.        , 0.        , 0.        , ..., 1.        , 1.        ,
        0.35422886],
       ...,
       [0.        , 1.        , 1.        , ..., 1.        , 0.66666667,
        0.11293532],
       [1.        , 1.        , 0.        , ..., 1.        , 1.        ,
        0.55870647],
       [0.        , 0.        , 0.        , ..., 1.        , 0.        ,
        0.86965174]])
```

In [99]:
```
x1_train,x1_test,y1_train,y1_test=train_test_split(x1_scaled,y,test_size=0.3,r
```

In [100…
```
for model in models:
  model.fit(x1_train,y1_train)
  y1_pred=model.predict(x1_test)
  print(classification_report(y1_test,y1_pred))
```

	precision	recall	f1-score	support
0	0.84	0.86	0.85	1585
1	0.55	0.51	0.53	528
accuracy			0.77	2113
macro avg	0.69	0.68	0.69	2113
weighted avg	0.77	0.77	0.77	2113

	precision	recall	f1-score	support
0	0.85	0.90	0.87	1585
1	0.62	0.51	0.56	528
accuracy			0.80	2113
macro avg	0.73	0.70	0.72	2113
weighted avg	0.79	0.80	0.79	2113

	precision	recall	f1-score	support
0	0.91	0.77	0.83	1585
1	0.52	0.76	0.62	528
accuracy			0.77	2113
macro avg	0.71	0.76	0.73	2113
weighted avg	0.81	0.77	0.78	2113

BEST MODEL

MODEL = AdaBoostClassifier

SCORE = 81%